



Knowledge-enhanced personalized hierarchical attention network for sequential recommendation

Shuqi Ruan¹ · Chao Yang¹ · Dongsheng Li²

Accepted: 19 December 2023 / Published online: 17 January 2024

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

Sequential recommendation aims to predict the next items that users will interact with according to the sequential dependencies within historical user interactions. Recently, self-attention based sequence modeling methods have become the mainstream method due to their competitive accuracy. Despite their effectiveness, these methods still have non-trivial limitations: (1) they mainly take the transition patterns between items into consideration but ignore the semantic associations between items, and (2) they mostly focus on dynamic short-term user preferences and fail to consider user static long-term preferences explicitly. To address these limitations, we propose a Knowledge Enhanced Personalized Hierarchical Attention Network (KPHAN), which can incorporate the semantic associations among items by learning from knowledge graphs and capture the fine-grained long- and short-term interests of users through a novel personalized hierarchical attention network. Specifically, we employ the entities and relationships in the knowledge graph to enrich semantic information for items while preserving the structural information of the knowledge graph. The self-attention mechanism then captures semantic associations among items to obtain short-term user preferences more accurately. Finally, a personalized hierarchical attention network is developed to generate the final user preference representations, which can fully capture user static long-term preferences while fusing dynamic short-term preferences. Experimental results on three real-world datasets demonstrate that our method can outperform prior works by 2.7% - 35.5% on HR metrics and 6.7% - 27.9% on NDCG metrics.

Keywords Knowledge graph · Sequential recommendation · Personalized hierarchical attention · Long- and short-term preferences

✉ Chao Yang
yangchaoedu@hnu.edu.cn

Shuqi Ruan
shuqiruan@hnu.edu.cn

Dongsheng Li
dongсли@microsoft.com

¹ Hunan University, NO. 2 Lushan South Road, Yuelu District, Changsha 410000, China

² Microsoft Research Asia, NO. 701 Yunjin Road, Xuhui District, Shanghai 200030, China

1 Introduction

Recommender system can assist users in obtaining the information they are genuinely interested in from the massive data, which is one of the most important applications in e-commerce platforms [1, 2]. Conventional recommender systems usually assume that user preferences are static. However, user preferences and item popularity are intrinsically dynamic and evolve over time [3]. Therefore, many sequential recommendation algorithms have been proposed during recent years, which can learn sequential patterns from user interaction sequences and capture user interest drifts over time to achieve more accurate recommendations [3–6].

Existing sequential recommendation methods could be divided into two main categories. The first category of methods adopts conventional techniques, such as Markov chain (MC) [7] and matrix factorization (MF) [8]. Markov chain-based methods mainly infer the future interests of users based on the most recent interactions. For instance, FPMC [7] fuses MF and MC for capturing low-order dependencies. FOSSIL [9] and FISM [10] combine similarity-based and higher-order Markov chain methods to learn more complex behavioral dependencies. However, this category of methods usually cannot sufficiently capture long-term dependencies due to the Markov property assumption of user interactions. The second category of methods is based on deep learning. For instance, Recurrent Neural Networks (RNNs) are prevalent in this category of methods due to their effective sequential modeling capability. Hidasi et al. [4] first utilized GRU to capture the sequential dependencies in sessions. CNN-based methods [11, 12] have also been widely used to capture local dependencies in user interaction sequences. However, both RNN-based and CNN-based methods are not capable of capturing long-term dependencies. Recently, the transformer has achieved state-of-the-art performance in machine translation tasks based on an attention mechanism called self-attention [13]. Inspired by the transformer, Wang et al. [5] first employed self-attention in sequential recommendation tasks and proposed the SASRec method, which can improve performance by learning from the long-term dependencies. And then, several self-attention based methods have been proposed [5, 14–19]. However, most of them ignore the auxiliary information, which is important in enhancing the performance of sequential recommendation as shown in recent works [14, 19–22]. In addition, GNN-based [23–26] methods have also achieved promising results, capturing more complex transitions of items and obtaining more accurate item embeddings to improve recommendation performance.

Although the above methods have achieved promising results, they still suffer from the following two limitations. Firstly, previous works incorporating auxiliary information mainly utilized some unstructured attributes of items, such as item category, brand, description, etc., or used knowledge graph as attribute information of items, failing to model the semantic associations between items explicitly. In fact, the semantic associations between items can play a crucial role in improving recommendation accuracy by better mining user preferences and promoting semantically associated items. As shown in Figure 1, *Avatar* has a strong semantic relationship with *Titanic* due to the same director but has a weak semantic relationship with *Brave Heart* due to no direct semantic relationship. According to this semantic relationship, the recommendation model should be more likely to recommend *Titanic* instead of *Brave Heart* at the next moment. Secondly, previous self-attention based works adopt the representation of the last time step as the final user embedding, which can accurately reflect the user dynamic short-term preference but may lose the user static long-term preference throughout the interaction sequence. At the same time, the previous work considered long-term user preferences mainly through directly initialized user embedding. In fact, the static

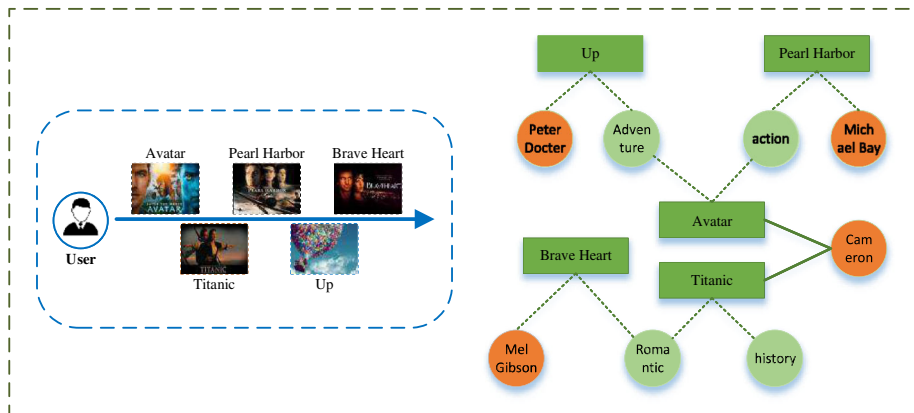


Figure 1 An illustrative example of semantic associations between items. The left part represents the user interaction sequence, and the right part represents the item knowledge graph of the interaction sequence. The green rectangles represent the five movies watched by the user, and the circles represent the semantic attributes of the movies. Here, the orange circles indicate director semantic relationship and the green circles indicate genre semantic relationship

long-term preferences of users in different subsequences are different, so it is challenging to obtain personalized static long-term user preferences by directly initializing user embedding.

To this end, we propose a Knowledge-enhanced Personalized Hierarchical Attention Network, namely KPHAN, which consists of a Knowledge-enhanced Dynamic Short-term Preference Module(KGSP) and Personalized Hierarchical Attention-based Long-term and Short-term Preference Fusion Module(LSPF). In the KGSP module, we introduce the knowledge graph of items to provide abundant semantic information through the knowledge-aware item encoding layer and lay the foundation for modeling semantic associations between items. Then, an adaptive attention module thoroughly mines item semantic associations to model dynamic short-term preferences. In the LSPF module, we apply the personalized hierarchical attention mechanism to capture necessary static long-term preferences. Then, we fuse user dynamic short-term preferences with static long-term preferences to obtain a more personalized user preference representation. Finally, we use the dot product between user embedding and item embedding as the recommendation score.

In summary, the main contributions of this paper are as follows:

- We propose a knowledge-enhanced personalized hierarchical attention network to exploit semantic transitions over sequence, which can provide personalized user preference modelling.
- We design a knowledge-aware item encoding module to model the semantic association between items and an adaptive attention module to capture user dynamic short-term preferences. In addition, we design a personalized hierarchical attention network to capture user static long-term preferences and fuse them with user dynamic short-term preferences.
- Experiments on three public datasets demonstrate that our method can significantly outperform state-of-the-art baselines in recommendation accuracy.

The rest of this paper is organized as follows. Section 2 summarizes the research works related to knowledge graph-based recommendation and sequential recommendation. Section 3 formulates the sequential recommendation problem and gives the background of

knowledge graph embedding. Section 4 presents the method proposed in this paper. Section 5 offers the experimental details. Section 6 summarizes the work of this paper.

2 Related work

2.1 Knowledge-based recommendation

In recent years, knowledge graph-based recommender systems [27] have attracted great interest from researchers, which can alleviate the problems of data sparsity, cold start, etc., while improving the accuracy, diversity, and interpretability of the recommender system. They could be mainly divided into three categories: embedding-based methods [28, 29], connection-based methods [30, 31], and propagation-based methods [32, 33].

DKN [28] and CKE [29] are classic embedding-based methods, which mainly preprocess the abundant semantic information of knowledge graphs by knowledge graph embedding (KGE) algorithms and utilize learned entity embeddings to enrich item and user representations. However, these methods are less intuitive and effective in characterizing higher-order semantic relationships between entities and are generally not interpretable. By manually defining meta-paths, connection-based methods [30] explore the benefits of various patterns of connections among items. KPRN [31] is a representative model that effectively infers user preferences by considering the semantic relations of paths. Specifically, generating representations of multiple user-items paths by combining the semantics of entities and relations, and a novel weighted pooling operation is applied to discriminate the strength of different paths. The rationale for inferring user-item interactions is to model sequential dependencies in paths by LSTM. However, they are not suitable for scenarios with sparse interaction data. Propagation-based methods [32, 33] combine the merits of the above methods to mine useful information from the knowledge graph but usually suffer from high computational costs.

The above methods are mainly used in conventional recommender systems, which ignore the order of historical interactions. Huang et al. [34] firstly integrate RNN-based networks with Key-Value Memory Network to incorporate knowledge information for sequential recommendation. Sequential preferences captured by RNN and attribute-level preferences learned by Key-Value Memory Network are combined into the final user preferences. And, Wang et al. [35] utilizes knowledge information in reinforcement learning-based sequential recommendation for the first time.

2.2 Sequential recommendation

Sequential recommendation aims to predict the next items that users will interact with according to the sequential dependencies within the users' historical interactions. The early works were mainly based on Markov Chain [36], which predicts the user interaction at the next moment by establishing a transition matrix. Rendle et al. [7] proposed the FPMC method by combining matrix factorization and Markov chain for the personalized sequential recommendation, which utilized pairwise interaction models to decompose Markov chain-based personalized transformation graphs. However, such low-order Markov chains cannot capture high-order complex dependencies containing multi-level cascading relationships across multiple interactions. Therefore, Kabbur et al. [10] and He et al. [9] proposed high-order Markov models named FOSSIL [9] and FISM [10], respectively, which combined similarity-based

methods and high-order Markov chain-based methods to capture high-order relationships among item sequences for sequential recommendation.

With the rapid development of deep learning [37], the recurrent neural network (RNN) is significantly effective for modeling data with sequential dependencies and has been successfully applied to sequential recommendation [4, 38, 39]. Hidasi et al. [4] first proposed the GRU4Rec, which utilizes GRU to model short-term user preferences based on user previous purchase behaviors. Many variants of RNN were proposed to improve the sequential recommendation performance [34, 35, 38, 40–43]. Hidasi et al. [38] proposed the P-RNN method to improve recommendation accuracy by utilizing rich item features (i.e., images and text). Inspired by the success of CNN in extracting image features, several methods [11, 12] proposed to embed user interaction sequences as images and model local sequential patterns through horizontal and vertical filters. Recently, self-attention-based methods [5, 14, 16, 18, 19, 34, 44–46] have made remarkable progress in modeling the long-distance dependencies for more accurate sequential recommendations. SASRec [5] adaptively assigns weights to previous items at each time step through a self-attention mechanism and seeks to identify items relevant to the next interaction from the user historical interaction sequences. BERT4Rec [15] exploits bidirectional self-attention to encode historical sequences. FDSA [14] adopts parallel self-attention to encode item sequences and side information sequences simultaneously. ICAI-SR [19] models item sequences and attribute sequences separately to predict the next item. In addition, GNN [23–26] has made some progress in modeling complex transitions of items. Wu et al. [23] constructed historical session sequences as directed graphs for the first time and then used GNN to capture complex transitions of items and generate accurate item embedding vectors. HG-GNN [26] learns long-term user preferences and item representations with rich semantics by constructing a novel heterogeneous graph. However, the above methods do not consider the semantic associations between items and pay little attention to static long-term user preferences, which cannot accurately and completely characterize user preferences.

Different from the above works, our method can model items with the help of item knowledge graphs, which are preprocessed by KGE algorithms [47, 48] to enrich semantic information of items and thus achieve more fine-grained item representations. Semantic associations between items are then captured via adaptive attention, which better captures sequential dependencies to model user dynamic short-term preferences while fusing static long-term preferences to achieve more accurate and complete user preference modeling.

3 Preliminaries

In this section, we first formulate the sequential recommendation problem and then introduce the background of knowledge graph embedding.

3.1 Problem formulation

The set of users is represented as $U = \{u_1, u_2, \dots, u_m\}$ and the set of items is represented as $V = \{v_1, v_2, \dots, v_n\}$. The interaction sequence for each user u is defined as $S^u = \{s_1^u, s_2^u, \dots, s_t^u\}$, sorted by ascending order of interaction time, in which s_t^u represents the item that user u interacted with at time t . Besides the user interaction sequences, we also take the knowledge graph information of all items as input. The knowledge graph comprises of entity set E , relation set R , and triplets represented by $(e_h, r, e_t) \in G$, where e_h and

e_t represent the head node and the tail node $e_h, e_t \in E$, corresponding to entities in the knowledge graph, and $r \in R$ defines the relationship between e_h and e_t . For example, *(Titanic, directedby, Cameron)* describes the director of the movie Titanic as Cameron. Assuming that item v in the interaction sequence can be linked to an entity $e_h \in E$ in the knowledge graph, we can get the related triplets of item v as illustrated above.

This work targets at the knowledge-enhanced sequential recommendation task with implicit feedback. The primary purpose is to predict the probability of the item that the user will likely interact with at time $t + 1$ based on the items interacted by the user before time t and the corresponding knowledge graph triplet information. The mathematical notation used in this paper is summarized in Table 1.

3.2 Knowledge graph embedding

The purpose of knowledge graph embedding is to encode entities and relationships in knowledge graphs into low-dimensional dense vectors while preserving knowledge graph structural information and semantic associations [47–50]. To facilitate the introduction of the knowledge-enhanced item representation in this work, we briefly introduce the translation-based KGE methods as follows.

- The basic idea of TransE [47] is to represent entities and relationships in the same space and wants $h + r \approx t$. The score function of TransE is as follows:

$$f_r(h, t) = \|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_2^2. \quad (1)$$

However, one disadvantage of TransE is its insufficient ability to deal with complex relationships [48].

- The main idea of the TransH method [48] is to project the head node h and the tail node t onto a hyperplane determined by w_r :

$$f_r(h, t) = \|\mathbf{h}_\perp + \mathbf{r} - \mathbf{t}_\perp\|_2^2, \quad (2)$$

Table 1 Notation definition and description

Notation	Description
U	User set
V	Item set
S^u	User u 's interactive sequence on item set
S_t^u	Items that user u interacts with at time t
E	The entity set of knowledge graph
G	The triplet set of knowledge graph
$\mathbf{e}_h$	Entity vector corresponding to item v
E_h	Entity vector set corresponding to item set
G_v	The first-order subgraphs of item v
E_v	The first-order neighbor item vectors of item v
E_{cv}	The contextual feature set for item set
$\tilde{\mathbf{V}}$	Knowledge-enhanced item sequence embedding
$\mathbf{p}_u^{long}$	Long-term preference
$\mathbf{p}_u^{short}$	Short-term preference

where $\mathbf{h}_\perp = \mathbf{h} - \mathbf{w}_r^T \mathbf{h} \mathbf{w}_r$ and $\mathbf{t}_\perp = \mathbf{t} - \mathbf{w}_r^T \mathbf{t} \mathbf{w}_r$ represent the hyperplane that maps the head and tail vectors to the corresponding relation, respectively.

- TransR [50] introduces a relationship transformation matrix to convert the entity into the corresponding relationship vector space. The score function in TransR is defined as follows:

$$f_r(h, t) = \|\mathbf{h}_r + \mathbf{r} - \mathbf{t}_r\|_2^2, \quad (3)$$

where $\mathbf{h}_r = \mathbf{M}_r \mathbf{h}$ and $\mathbf{t}_r = \mathbf{M}_r \mathbf{t}$ represent the mapping of entity vectors into relational space.

- TransD [49] maps the head entity and the tail entity into the relation space through two projection matrices $\mathbf{M}_r^1$ and $\mathbf{M}_r^2$, respectively, and the score function of TransD is defined as

$$f_r(h, t) = \|\mathbf{h}_r + \mathbf{r} - \mathbf{t}_r\|_2^2, \quad (4)$$

where $\mathbf{M}_r^1 = \mathbf{w}_r \mathbf{w}_h^T + \mathbf{I}$, $\mathbf{M}_r^2 = \mathbf{w}_r \mathbf{w}_t^T + \mathbf{I}$, $\mathbf{I}$ represents the identity matrix, and $\mathbf{h}_r = \mathbf{M}_r^1 \mathbf{h}$, $\mathbf{t}_r = \mathbf{M}_r^2 \mathbf{t}$.

In order for the above four models to correctly distinguish between positive and negative triplets, they all adopt the loss function of negative sampling and maximum interval strategy as follows:

$$\mathcal{L}(h, r, t) = \max(0, d_{pos} - d_{neg} + \text{margin}), \quad (5)$$

where d_{pos} is the score of a positive triplet, d_{neg} is the score of a negative triplet, and *margin* represents the maximum margin.

4 Method

In this section, we introduce our proposed KPHAN method. As shown in Figure 2, KPHAN includes two key components: 1) a knowledge-enhanced dynamic short-term preference (KGSP) module and 2) a personalized hierarchical attention-based long-term and short-term preference fusion (LSPF) module.

In the KGSP module, we adopt knowledge graph embedding algorithms to encode the corresponding entities of items and the contexts of the entities. Then, we design a bidirectional knowledge fusion method to enhance item representation. Finally, the item sequence is modeled through the self-attention mechanism to capture the long-distance dependency for enhancing the user dynamic short-term preferences modeling.

In the LSPF module, we design the personalized hierarchical attention mechanism to capture static long-term preferences while fusing dynamic short-term preferences to obtain the implicit user preference feature. Finally, we combine explicit and implicit user preferences as the final user preferences to predict future user interactions.

4.1 Knowledge-enhanced dynamic short-term preference modeling

4.1.1 Knowledge representation modeling

Auxiliary information is essential in solving the cold start problem of recommender systems. Many prior works have integrated auxiliary information into recommendation models [14, 34]. Zhang et al. [14] first proposed incorporating heterogeneous item attribute information

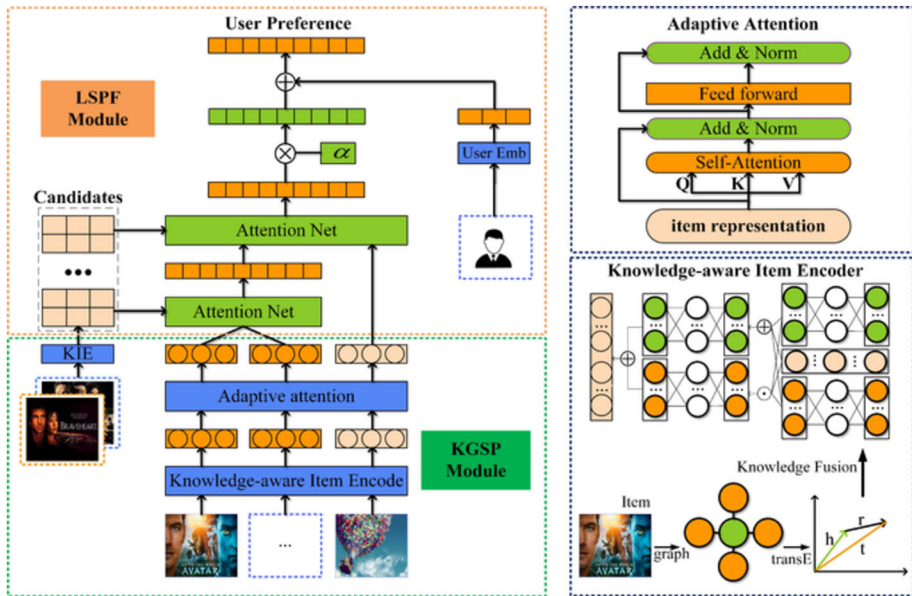


Figure 2 The overall architecture of the proposed model KPHAN

into sequence modeling. However, they are limited by item attributes in the dataset and ignore semantic associations between items. Therefore, we adopt knowledge graph embedding algorithms to enrich the semantic representation of items.

We fuse the knowledge information into item representations to enrich the representation of items and strengthen the semantic association between items. Specifically, we first adopt the entity linking method described in KB4Rec [51] to find the entity $e_h \in E$ corresponding to item v in the knowledge graph. Then, starting from the entity e_h , we find the first-order graph of the entity, $G_v \subset G$, where G_v represents the first-order subgraph of item v . Finally, the first-order subgraph is embedded through the translation-based methods to obtain the low-dimensional dense vector $\mathbf{e}_h \in E_h$ of the entity corresponding to the item v and the first-order neighboring entities vector $\mathbf{E}_v \in \mathbb{R}^{|G_v|-1 \times d}$ of the item v , where $E_h \in \mathbb{R}^{|V| \times d}$ represents the entity vector set corresponding to the item set.

We define the first-order neighboring entities as the contextual entities in this paper, which can provide helpful auxiliary information and semantic associations for the entity and help achieve more fine-grained item representation. The contextual entities of item v are defined as:

$$\text{context}(v) = \{\mathbf{e}_v | \mathbf{e}_v \in E_v\}. \quad (6)$$

Given the contextual entities of entity e , the context embedding is calculated as the average of the contextual entities vectors:

$$\mathbf{e}_{cv} = \frac{1}{|\text{context}(v)|} \sum_{e_v \in \text{context}(v)} \mathbf{e}_v, \quad (7)$$

where $\mathbf{e}_{cv} \in E_{cv}^{|V| \times d}$ is the contextual feature matrix of the contextual entities corresponding to the item.

4.1.2 Dynamic short-term preference modeling

Similar to Wang et al. [5], we employ an adaptive attention mechanism to encode sequences to capture short-term user preferences.

Knowledge-aware Item Encoder. We create an item embedding matrix $\mathbf{M} \in \mathbb{R}^{|V| \times d}$, where d represents the item embedding dimension, and $|V|$ represents the number of items. For modelling each sequence, we retrieve the corresponding input item embedding matrix $\mathbf{E} \in \mathbb{R}^{n \times d}$. We add a learnable position embedding $\mathbf{P} \in \mathbb{R}^{n \times d}$ to each sequence. The final embedding of an item sequence is $\hat{\mathbf{E}} = \mathbf{E} + \mathbf{P}$. Wang et al. [5] directly use $\hat{\mathbf{E}}$ as the input of the adaptive attention module. However, only using item embedding ignores the semantic association between items. Therefore, to enrich the representation of the item and strengthen the semantic association, we propose the bidirectional fusion $f_{Bi-interaction}$ to fuse the final embedding of item sequence $\hat{\mathbf{E}}$ with the knowledge graph embeddings. Then we obtain the knowledge-enhanced item sequence embedding $\tilde{\mathbf{V}} \in \mathbb{R}^{n \times d}$ and apply a dropout layer over it to alleviate the overfitting problem. The design of bidirectional fusion improves the model's ability to model nonlinearity. At the same time, the method utilizes the complementarity between features to fuse the advantages of each feature, thereby improving the model's performance. The details are described as follows:

$$\tilde{\mathbf{V}} = f_{Bi-interaction}(\hat{\mathbf{E}}, \hat{\mathbf{E}}_h, \hat{\mathbf{E}}_{cv}), \quad (8)$$

$$\hat{\mathbf{E}}_h = \tanh(\mathbf{W}\tilde{\mathbf{E}}_h + \mathbf{b}), \hat{\mathbf{E}}_{cv} = \tanh(\mathbf{W}\tilde{\mathbf{E}}_{cv} + \mathbf{b}), \quad (9)$$

$$f_{Bi-interaction} = \text{LeakyReLU}(\mathbf{W}_1(\hat{\mathbf{E}} + \hat{\mathbf{E}}_h + \hat{\mathbf{E}}_{cv})) \\ + \text{LeakyReLU}(\mathbf{W}_2(\hat{\mathbf{E}} \odot \hat{\mathbf{E}}_h \odot \hat{\mathbf{E}}_{cv})), \quad (10)$$

where $\tilde{\mathbf{E}}_h \in \mathbb{R}^{n \times d}$ is the entity embedding matrix corresponding to the item sequence $\mathbf{E}$, $\tilde{\mathbf{E}}_{cv} \in \mathbb{R}^{n \times d}$ is the context embedding matrix corresponding to the item sequence $\mathbf{E}$. $\tilde{\mathbf{E}}_h$ and $\tilde{\mathbf{E}}_{cv}$ are retrieved from $\mathbf{E}_h$ and $\mathbf{E}_{cv}$. $\hat{\mathbf{E}}_h$ and $\hat{\mathbf{E}}_{cv}$ project the context embedding and the entity embedding corresponding to $\mathbf{E}$ to the same space. $\mathbf{W}$, $\mathbf{W}_1$, $\mathbf{W}_2$, and $\mathbf{b}$ are learnable parameters, $\odot$ denotes the element-wise product.

Short-term Preference Modeling. The knowledge-enhanced item sequence embedding $\tilde{\mathbf{V}} \in \mathbb{R}^{n \times d}$ is used as the input to adaptive attention block to capture semantic associations between items while modeling user short-term interest preferences. Specifically, the adaptive attention block includes self-attention layer and feed-forward network (FFN). The self-attention layer adopts a scalar dot product calculated as follows:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}, \quad (11)$$

$$\mathbf{S} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}), \quad (12)$$

where $\mathbf{Q} = \text{LayerNorm}(\tilde{\mathbf{V}})\mathbf{W}^Q$, $\mathbf{K} = \tilde{\mathbf{V}}\mathbf{W}^K$, and $\mathbf{V} = \tilde{\mathbf{V}}\mathbf{W}^V$. $\mathbf{K}$ denotes keys, $\mathbf{Q}$ denotes queries, $\mathbf{V}$ denotes values, and $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V \in \mathbb{R}^{d \times d}$ are all learnable parameters. Layer-Norm represents layer normalization. It is worth noting that in order to ensure the temporal order of the sequence, i.e., the causality of the sequence, we adopt the mask mechanism when calculating $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$. For $\mathbf{Q}_i$ and $\mathbf{K}_j$ satisfying $i < j$, the attention weights between $\mathbf{Q}_i$ and $\mathbf{K}_j$ are calculated as the dependency between item i and item j , and the order change of the sequence is captured by calculating the attentional weighted sum of previous items.

To add non-linearity to the model, a feed-forward network is applied after the self-attention layer to obtain non-linear features. We use a two-layer feed-forward network as follows:

$$\mathbf{F} = \text{FFN}(\mathbf{S}) = \text{ReLU}(\text{Dropout}(\text{ReLU}(\text{Dropout}(\mathbf{S}\mathbf{W}^{(1)} + \mathbf{b})))\mathbf{W}^{(2)} + b)), \quad (13)$$

where Dropout means randomly dropout neurons with a certain probability during linearity. We use residual connections in the self-attention module and the feed-forward network.

To learn more complex sequential dependencies, we stack the adaptive attention block (i.e., self-attention layer and feed-forward network) ω times, and the b -th block ($b > 1$) is defined as follows:

$$\mathbf{S}^{(b)} = \text{Attention}(\mathbf{F}^{(b-1)}), \quad (14)$$

$$\mathbf{F}^{(b)} = \text{FFN}(\mathbf{S}^{(b)}), \quad (15)$$

when $b = 1$, $\mathbf{S} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$, and $\mathbf{F} = \text{FFN}(\mathbf{S})$. $\mathbf{F}_n^{(\omega)}$ is the feature vector of the last item in the last block, which combines the features of all previous items to obtain a more accurate representation of the current item. The current item representation is vital for user preference prediction, so we regard $\mathbf{P}_u^{\text{short}} = \mathbf{F}_n^{(\omega)}$ as the short-term user preference embedding.

4.2 Personalized hierarchical attention-based long-term and short-term preference fusion

In addition to the short-term user preference, static long-term preference also plays an indispensable role in predicting the user's next item interaction. This section mainly introduces how to obtain the static long-term user preference and how to balance between static long-term preferences and dynamic short-term preferences.

4.2.1 Static long-term preference modeling

Although $\mathbf{P}_u^{\text{short}}$ combines the features of all previous items in the adaptive attention module, this can only represent the user's preference features at the current moment, and the implicit static long-term preference is easily ignored by short-term preference modeling. Assuming that items similar to the candidate item appear many times in the historical sequence, the user is very likely to purchase the candidate item, which plays an important role in mining the user static long-term preferences implied in the historical sequence to enhance user interest preferences and improve recommendation accuracy. We demonstrate the usefulness of mining static long-term preferences in Section 5.4.

We calculate the similarity between $F_1^{(\omega)}$, $F_2^{(\omega)}$, ..., $F_{n-1}^{(\omega)}$ and the candidate item to obtain the static long-term user preference. The similarity between the historical item and the candidate item at time t is calculated as follows:

$$\alpha_t^u = \text{softmax}(\mathbf{F}_t^{(\omega)\top} q) = \frac{\exp(\mathbf{F}_t^{(\omega)\top} q)}{\sum_{i=1}^{n-1} \exp(\mathbf{F}_i^{(\omega)\top} q)}, \quad (16)$$

where q is the candidate item and α_t^u is the user attention score of the item at time t , the static long-term user preference is the weighted sum of the embeddings from the previous $n - 1$ timestamps as follows:

$$\mathbf{P}_u^{\text{long}} = \sum_{t=1}^{n-1} \alpha_t^u \cdot \mathbf{F}_t^{(\omega)}. \quad (17)$$

4.2.2 Long-term and short-term preference fusion

Long-term and short-term preferences play different roles in predicting the next item, so we further explore the varying importance of long-term and short-term preferences in sequential recommendation. Specifically, we propose the long-term and short-term attention mechanism as follows:

$$\begin{aligned}\beta_u^{short} &= \text{softmax}(\mathbf{P}_u^{short^T} \mathbf{q}) \\ &= \frac{\exp(\mathbf{P}_u^{short^T} \mathbf{q})}{\exp(\mathbf{P}_u^{short^T} \mathbf{q}) + \exp(\mathbf{P}_u^{long^T} \mathbf{q})},\end{aligned}\quad (18)$$

$$\begin{aligned}\beta_u^{long} &= \text{softmax}(\mathbf{P}_u^{long^T} \mathbf{q}) \\ &= \frac{\exp(\mathbf{P}_u^{long^T} \mathbf{q})}{\exp(\mathbf{P}_u^{short^T} \mathbf{q}) + \exp(\mathbf{P}_u^{long^T} \mathbf{q})},\end{aligned}\quad (19)$$

$$\mathbf{P}_u^{implicit} = \beta_u^{long} \cdot \mathbf{P}_u^{long} + \beta_u^{short} \cdot \mathbf{P}_u^{short}, \quad (20)$$

where $\mathbf{P}_u^{implicit}$ denotes implicit user preferences, consisting of long-term and short-term preferences. β_u^{short} and β_u^{long} are the weights of short-term and long-term preferences, respectively, and $\mathbf{q}$ is the embedding of the next item to be interacted with.

We introduce an explicit user preference matrix $\mathbf{P}_u^{explicit} \in |u| \times d$ learned from scratch to provide personalized recommendation, where d represents the dimension and $|u|$ represents the number of users. A weight α is used to balance the implicit and explicit user preferences to obtain the final user preference representation $\mathbf{P}_u \in |u| \times d$ as follows:

$$\mathbf{P}_u = \alpha \mathbf{P}_u^{implicit} + \mathbf{P}_u^{explicit}. \quad (21)$$

4.3 User preference prediction and model training

4.3.1 User preference prediction

To predict the next item, we compute the dot product of the final user preference representation and the set of candidate itemsets to obtain the probability of the user interacting with the next item as follows:

$$\hat{y} = \mathbf{P}_u^i \mathbf{N}^T, \quad (22)$$

where $\mathbf{N}^T \in \mathbb{R}^{d \times n}$ represents 101 candidate items consisting of a ground truth item and 100 negative items, which are obtained by the Knowledge-aware Item Encoder module, and $\hat{y}$ is the predicted score of the i -th user on the candidate items. Higher scores indicate a higher probability of the user interacting with the candidate items. We sort the candidate items by the scores in a descending order and select the top K items as the recommended items.

4.3.2 Model training

To resolve the maximum sequence length restriction and preserve all training data, different from the *shifted* model training method in the SASRec method [5], we make a finer division of user sequences. Specifically, we set the maximum length of the interaction sequence in the model as n and take a sliding window with a step size of 1 to select sub-sequences with size n

on the entire user interaction sequence. For each sub-sequence with size n , the corresponding $(n + 1)$ th item is treated as the ground truth item for prediction. If the length of the interaction sequence is less than n , we pad zeros to the left of the sequence. Based on this, we design a sequence-aware binary cross-entropy loss function as follows:

$$\mathcal{L} = - \sum_{i \in U} \sum_{j=1}^k (\log \hat{y}_{i,j} + \sum_{j^- \in V_{(i,j)}^-} \log(1 - \hat{y}_{i,j^-})) + \lambda \mathcal{L}_{reg}, \quad (23)$$

where k is the maximum number of sequences that can be divided by user i , j is the positive sample of each divided sequence, and $V_{(i,j)}^-$ is the set of negative samples sampled for a positive sample j of user i . We sample 100 negative samples for each positive sample. $\mathcal{L}_{reg}$ is the regularization term for all parameters of the model. We utilize Adam [52] to minimize the above loss function over mini-batches for model training.

5 Experiment

In this section, we evaluate our proposed method KPHAN on three datasets: Movielens-20M, Movielens-1M, and LastFM. We aim to answer the following research questions:

- **RQ1:**How does KPHAN perform compared with state-of-the-art sequential recommendation methods?
- **RQ2:**How do different components affect KPHAN?
- **RQ3:**How does knowledge graph information affect the KPHAN model?

5.1 Dataset description

Movielens: Movielens [53] is a benchmark dataset for evaluating collaborative filtering algorithms. This paper selects two datasets with different scales, Movielens-20M and Movielens-1M, containing 20 million and 1 million interactions, respectively.

LastFM: The LastFM dataset was proposed for music recommendation, which contains about 1 billion listening events. We select the data from 2005 to 2009 according to the timestamps.

To ensure data quality, we adopt 5-core items and filter out users with less than 20 interactions or more than 300 interactions. According to the data pre-processing method in [5], we regard ratings as implicit feedback (i.e., all items that users have interacted with are considered positive samples, and items that have not been interacted with are regarded as negative samples). The datasets are sorted in time order by interaction timestamps. We hold out the last item for each user as the test data and the rest as training data. In the training data, the training sequence length is constructed according to the average interaction length of the dataset. We randomly sample 100 negative samples for each positive sample in the test stage.

For the knowledge graph dataset, we choose Freebase [54]. In order to link the items in the recommendation datasets to the entities in the knowledge graph, we adopt the method proposed in KB4Rec [51] and select the first-order subgraph of the linked entities as the item knowledge graph. Since there are many useless relationships, we filter out the relationships with less than 6000 occurrences and only select the important relationships from them. The detailed statistics of the datasets are shown in Table 2.

Table 2 Statistics of the datasets

Datasets	Movielens-20M	Movielens-1M	LastFM
#Users	97,112	5,074	5,853
#Items	9,517	3,143	43,942
#Interactions	7,312,639	497,134	423,124
Avg. seq. len.	75	98	73
#Link Entities	9,475	3,121	24,493
#Link rate	99.60%	99.30%	55.70%
#Relations	18	18	23

5.2 Experimental setup

5.2.1 Evaluation metrics

To evaluate the recommendation performance, we adopt two commonly used evaluation metrics in terms of top-K recommendation: Hit Rate (HR@K) and Normalized Discounted Cumulative Gain (NDCG@K), following the settings in [5]. HR@K indicates that the Hit Rate is 1 if the ground truth item appears in the top K items and 0 otherwise. NDCG@K is an evaluation metric that considers the position of correct recommendation, and higher ranking of the ground truth item will have greater NDCG value. Assuming that the ranking of the user's ground truth item is $rank_u$ and $rank_u \leq K$, then the NDCG is calculated as follows: $\frac{1}{\log_2(rank_u + 1)}$, if $rank_u > K$, then NDCG is 0.

5.2.2 Baselines

To demonstrate the effectiveness of the proposed KPHAN method, we compare KHAN with three kinds of baselines.

The first kind of methods are traditional recommendation methods:

- **PopRec** ranks items based on their popularity (the number of times that an item appears in the training set).
- **Bayesian Personalized Ranking (BPR)** [55] targets at user implicit feedback and adopts the pair-wise loss function to factorize the user-item interaction matrix.

The second kind of methods are sequential recommendation algorithms based on a first-order Markov chain:

- **Factorized Personalized Markov Chains (FPMC)** [7] combines matrix factorization with first-order Markov chains, capturing item-to-item transitions and long-term user preferences.
- **Translation-based Recommendation (TransRec)** [56] embeds the user and item into the translation space and recommends the next item by computing the sum of the translation vector of the previous item and user.

The third kind of methods are sequential recommendation algorithms based on deep learning techniques:

- **GRU4Rec** [4] is the first application of GRU to session-based recommendations to capture users' long-term dependencies.

- **KSR** [34] proposes combining knowledge graph information and memory network for sequential recommendation at the first time.
- **SASRec** [5] is the first to introduce self-attention mechanism into sequential recommendation task, achieving the state-of-the-art performance.
- **FDSA** [14] is based on SASRec [5] and adopts self-attention to learn transitions between items, and transitions between heterogeneous features of items.
- **HGN** [57] applies the hierarchical gating network to capture the short-term interests and adopts matrix factorization to capture the long-term interest.
- **KERL** [35] introduces knowledge information to the reinforcement learning-based sequential recommendation for the first time.
- **ICAI-SR** [19] directly models the complex relations between items and categorical attributes.
- **HG-GNN** [26] learns long-term user preferences and item representations through a global heterogeneous graph.

5.2.3 Experiment settings

We use PyTorch to implement our method. The learning rate is 0.001 on the three datasets, and the regularization coefficient is set to 0.0001. The dropout rate is set to 0.7 on the Movielens-20M dataset and 0.5 on the other two datasets. The knowledge embedding method is transR on the Movielens-1M dataset and TransE on the other two datasets. The knowledge embedding dimension and item embedding dimension are 50, the self-attention module is stacked in two layers, and the training sequence length is 100. The batch size is 2048 on the Movielens-20M dataset and 1024 for the remaining two datasets.

5.3 Overall performance comparison (RQ1)

The recommendation performance of different methods is compared in Table 3. Firstly, it can be seen from the results that our method achieves the best performance on both evaluation metrics, which demonstrates the effectiveness of the proposed knowledge-enhanced personalized hierarchical attention network. The proposed knowledge-enhanced item encoding module provides semantic associations between items, which make the recommended items be more suitable for the context. The personalized hierarchical attention network fuses users' long-term and short-term preferences and obtains more accurate and comprehensive personalized user embeddings.

Secondly, the sequential recommendation methods that consider the order of user interactions can bring more accurate recommendation than the conventional recommendation methods that do not consider the interaction order. The recommendation methods, FDSA, ICAI-SR, KSR, and KERL, integrate attributes or knowledge information into SASRec and GRU methods. FDSA, KSR, and KERL are better than their basic methods in the two evaluation metrics, indicating that attribute or knowledge information is indispensable in improving recommendation performance. ICAI-SR utilizes different types of category attribute information, but the performance is degraded, which may be due to the noise brought by its heterogeneous graph aggregation on attribute nodes. Compared to the best performing models (i.e., KSR, FDSA, and KERL) that incorporate knowledge or attributes on each dataset, the Hit Rate of our proposed KPHAN method has been improved by an average of 15.2%, and an average of 16.8% increase on the NDCG. The reason is that our method not only captures

Table 3 Performance comparison of each method

Datasets	Metric	Pop	BPR-MF	FPMC	TransRec	GRU4Rec	KSR	SASRec	FDSA	HGN	KERL	ICAI-SR	HG-GNN	KPHAN	Improv.	p-value
Movielens-20M	NDCG@5	0.3217	0.4377	0.4489	0.4546	0.4626	0.4712	0.4370	0.4613	<u>0.4745</u>	0.3751	0.2966	0.2298	0.5475	15.4%	0.0009792
	NDCG@10	0.3761	0.4890	0.5014	0.4811	0.4837	0.5152	0.5212	<u>0.5402</u>	0.5265	0.4309	0.3529	0.2976	0.5913	9.5%	0.0004035
	NDCG@15	0.4033	0.5071	0.5216	0.5138	0.5071	0.5287	0.5169	<u>0.5146</u>	<u>0.5437</u>	0.4536	0.3812	0.3270	0.6049	11.3%	0.0006413
	HR@5	0.4666	0.6560	0.6379	0.6582	0.6140	0.6631	0.6193	0.6427	<u>0.6726</u>	0.5449	0.4399	0.3850	0.7345	9.2%	0.0007979
	HR@10	0.6352	0.7640	0.8017	0.8052	0.7787	0.8014	0.8213	0.8224	<u>0.8320</u>	0.7172	0.6145	0.5950	0.8691	4.5%	0.0012977
	HR@15	0.7378	0.8324	0.8751	0.8739	0.8602	0.8574	0.8717	0.8774	<u>0.8965</u>	0.8029	0.7210	0.7060	0.9203	2.7%	0.0002509
Movielens-1M	NDCG@5	0.2021	0.3671	0.4364	0.4228	0.4357	0.4198	0.3679	0.3540	0.3102	<u>0.4758</u>	0.1742	0.1449	0.5083	6.8%	0.0486128
	NDCG@10	0.2488	0.4235	0.4852	0.4734	0.4590	0.4896	0.4597	0.4463	0.3678	<u>0.5137</u>	0.2224	0.2519	0.5514	7.3%	0.0277766
	NDCG@15	0.2749	0.4479	0.5045	0.4929	0.4896	0.5023	0.4463	0.4321	0.3915	<u>0.5305</u>	0.2484	0.2885	0.5662	6.7%	0.0230278
	HR@5	0.3100	0.5252	0.5948	0.5907	0.6480	<u>0.6501</u>	0.5278	0.5028	0.4540	0.6246	0.2747	0.2560	0.6803	4.7%	0.0002316
	HR@10	0.4553	0.6996	0.7448	<u>0.7473</u>	0.7154	0.7323	0.7268	0.7156	0.6328	0.7439	0.4241	0.5910	0.8086	8.2%	2.988E-05
	HR@15	0.5544	0.7913	0.8181	0.8210	0.8185	<u>0.8246</u>	0.7759	0.7621	0.7223	0.8041	0.5227	0.7290	0.8689	5.4%	0.0001694
LastFM	NDCG@5	0.0988	0.1249	<u>0.1489</u>	0.1077	0.0701	0.0854	0.1099	0.0914	0.1286	0.0928	0.0942	0.0247	0.1906	27.9%	2.129E-05
	NDCG@10	0.1248	0.1610	<u>0.1811</u>	0.1239	0.0854	0.1394	0.1426	0.1148	0.1593	0.1171	0.1211	0.0304	0.2249	24.2%	3.774E-05
	NDCG@15	0.1410	0.1887	<u>0.2013</u>	0.1338	0.0959	0.1624	0.1630	0.1271	0.1767	0.1337	0.1362	0.0392	0.2429	20.7%	0.0004118
	HR@5	0.1478	0.2118	<u>0.2142</u>	0.2049	0.1307	0.1432	0.1666	0.1328	0.1875	0.1348	0.1411	0.0510	0.2903	35.5%	8.118E-05
	HR@10	0.2288	0.2690	<u>0.3140</u>	0.3061	0.2090	0.2531	0.2619	0.2051	0.2825	0.2108	0.2248	0.0680	0.4048	28.9%	0.0027243
	HR@15	0.2901	0.3449	<u>0.3904</u>	0.3887	0.2722	0.2854	0.3379	0.2654	0.3487	0.2738	0.2819	0.1030	0.4639	18.8%	0.000229

Best performance in bold, Secondary performance is underlined, %Improv. represents the improvement of our method over the best baseline, and p-value denotes the significant difference for t-test

sequence transitions of items and semantic associations between items, but also takes into account both long-term and short-term preferences, which leads to better performance.

Finally, both HGN and HG-GNN explicitly model the user long-term preferences. And it is evident that HGN, which considers both long-term and short-term preferences, is basically better than other baselines which do not consider both long-term and short-term preferences. Still, it only models the features of items but ignores the semantic association between items, so the recommendation accuracy is lower than our proposed method KPHAN. HG-GNN encodes user long-term preferences by constructing a global heterogeneous graph. However, the sequential recommendation task treats the user history sequence as a session so that each user has only one connected edge with the sequence when constructing the global heterogeneous graph, which hinders accurately encoding long-term user preferences. Besides, multi-layer graph aggregation brings noisy information that degrades performance. In contrast, our method models different subsequences of user interactions more accurately through an attention mechanism and a sliding window training format to obtain personalized long-term user preferences, which significantly improves recommendation accuracy.

5.4 Ablation study (RQ2)

To verify the effectiveness of the knowledge-enhanced item representation module and the personalized hierarchical attention module, we conduct ablation studies on three variants of KPHAN, as shown in Table 4. KPHAN-K&A represents the basic model that ignores the knowledge enhancement module and the personalized hierarchical attention module, which is different from the SASRec method only in the model training. Its performance is higher than that of SASRec on the Movielens datasets (see Table 3), indicating that the proposed sliding window-based training is better than the *shifted* training method in SASRec.

KPHAN-A explores the usefulness of knowledge graph without personalized hierarchical attention. The effect of the basic model is significantly improved after adding knowledge information. On the three different datasets, the Hit Rate increased by an average of 7.8%, and the NDCG increased by an average of 7.7% compared with KPHAN-K&A. KPHAN-K explores the usefulness of hierarchical attention by removing the knowledge enhancement module. The Hit Rate on three different datasets has increased by an average of 1.4%, and the NDCG has increased by an average of 2.4% compared with KPHAN-K&A. Compared with all three variants, the performance of KPHAN is significantly higher. Our knowledge enhancement module and personalized hierarchical attention module complement each other and enable the model to achieve the best performance. KPHAN can exert a strong performance advantage even on the LastFM dataset with a moderate entity link rate.

Table 4 Ablation study of KPHAN on three datasets

Variants	Movielens-20M		Movielens-1M		LastFM	
	HR@10	NDCG@10	HR@10	NDCG@10	HR@10	NDCG@10
KPHAN-K&A	0.8601	0.5729	0.7816	0.5350	0.2112	0.1138
KPHAN-A	0.8626	0.5783	0.7956	0.5393	0.3381	0.1993
KPHAN-K	0.8647	0.586	0.7941	0.5454	0.2192	0.1199
KPHAN	0.8691	0.5913	0.8086	0.5514	0.4048	0.2249

5.5 Sensitivity analysis of KPHAN (RQ3)

To further understand the impacts of knowledge graph information on the model, we first test the impact of the knowledge graph embedding methods on performance. Then we analyze the effects of the relation number in the knowledge graph on the model performance. Finally, we study the effect of different knowledge fusion methods on the model performance.

5.5.1 Effect of knowledge graph embedding

We tested the effect of four KGE methods, such as TransE, TransH, TransR and TransD on the models KPHAN and KPHAN-A (i.e., Removing personalized hierarchical attention from our model KPHAN to explore the effect of different KGE methods on the Knowledge-aware Item Encoder module) on the Movielens-1M and LastFM datasets, and the results are shown in Figure 3.

The results on the Movielens-1M dataset are shown in Figure 3(a). The KPHAN model has little change in the performance of NDCG evaluation metrics on transH, transR and transD methods. On the HR metrics, the encoding using the transR method is significantly higher than the other three methods, which may be due to the fact that the representation of the same entity in the movie dataset under different relationships is widely disparate, so the

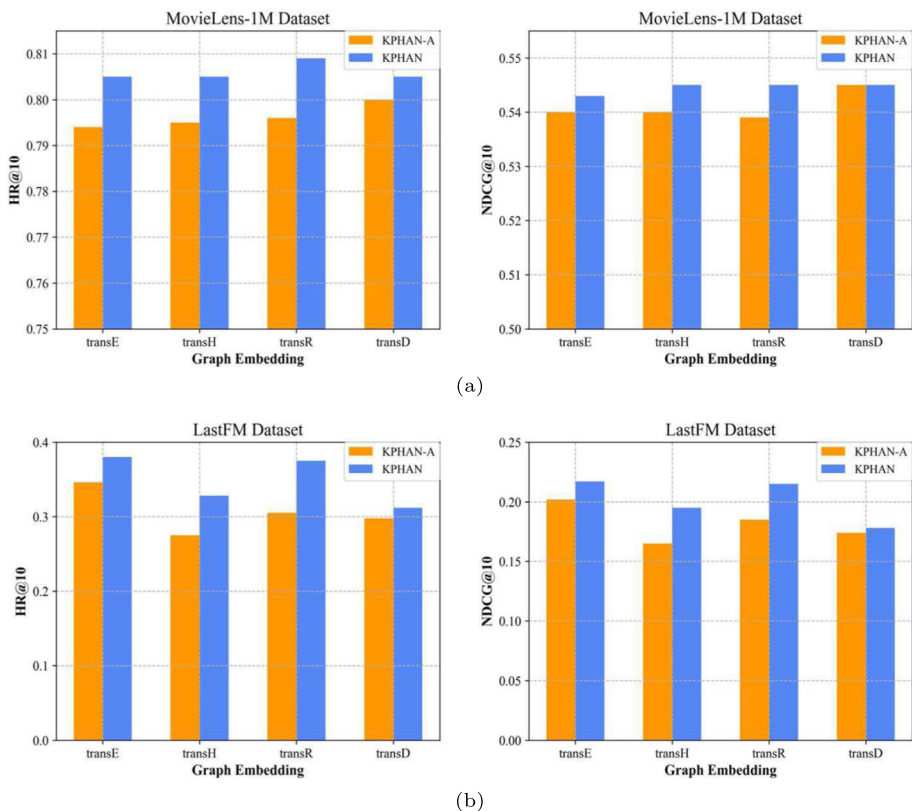


Figure 3 Effects of knowledge graph embedding

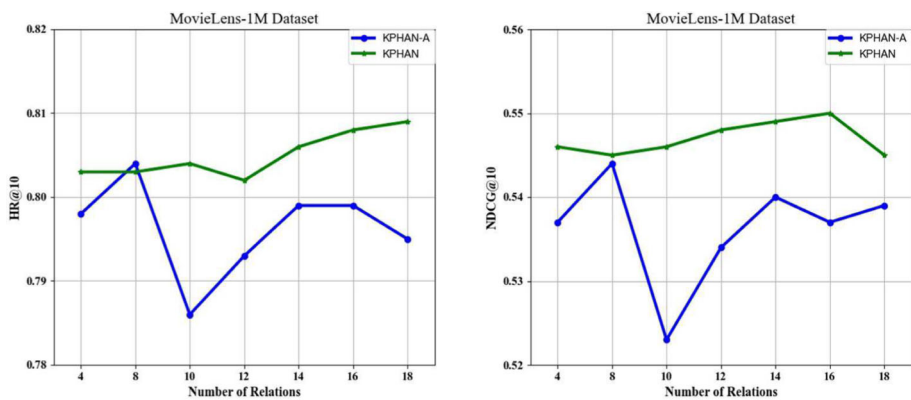
complex KGE encoding method is more beneficial to obtain more accurate entity encoding. Therefore, we use the tranR method to encode the entities in the movie knowledge graph.

The results of the LastFM dataset are shown in Figure 3(b). On the two evaluation metrics of HR and NDCG, the performance of using TransE is comparable with that of using TransR. However, KPHAN-A performs better when using the transE method than transR, which may be due to the fact that the same entity under different relationships rarely exists in the music dataset, so a better performance can be obtained by using the simple transE method. Instead, using a complex method to encode the knowledge graph leads to model overfitting. Meanwhile, the simple knowledge embedding method has less computational overhead. So we encode the entities using the transE method for the LastFM dataset.

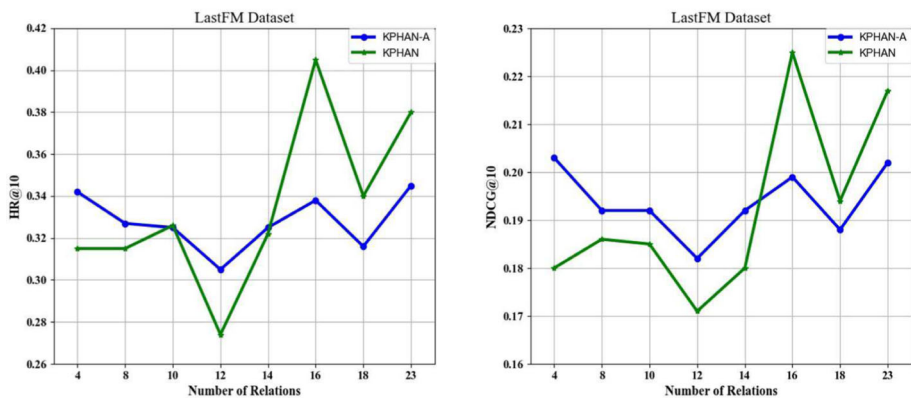
5.5.2 Effect of number of relations in knowledge graph

We further explore the effect of the number of first-order subgraph relations on KPHAN according to the KGE method identified in Section 5.5.1, as shown in Figure 4.

The results of the Movielens-1M dataset are shown in Figure 4(a). The model achieves the best performance on NDCG when the number of relationships is 16. There is little difference



(a)



(b)

Figure 4 Effect of number of relations in knowledge graph

between 16 and 18 in the number of relationships in HR. Therefore, the optimal number of relations for the Movielens dataset is 16.

The results of the LastFM dataset are shown in Figure 4(b). The best performance is obtained when the number of relations is 16. The performance of KPHAN is inferior to that of the KPHAN-A variant when the number of relations is less than 16. We think that this result may be due to the not high link rate of entities in the LastFM dataset. Moreover, few relationships make it difficult for the hierarchical attention module to distinguish the importance of different items and bias the model in static long-term preferences. Meanwhile, when the number of relations exceeds 16, the model performance also deteriorates because too many relationships may introduce noise into the model, reducing the performance. A common trend in both datasets is that the number of relations is not positively correlated with the recommendation performance. Different combinations of relations have specific effects, and we will explore how to find the best combination of relations in the future.

5.5.3 Effect of knowledge fusion methods

We further study the effectiveness of different knowledge fusion methods in the knowledge-enhanced item encoding module based on graph embedding methods and the number of relationships determined above. The results are given in Figure 5, where the *entity* and *context* denote that fusion with the item entity embedding and fusion with the context embedding, respectively. The *entity+context* and *item+entity+context* denote that bi-interaction fusion with item entity and context embedding and bi-interaction fusion with item, entity, and context embedding, respectively.

From the results in Figure 5, the bi-interactive fusion method of *item+entity+context* achieves the best performance on Movielens-1M and LastFM datasets. Meanwhile, KPHAN performs best when fusing only item-entity embeddings on Movielens-20M. The Movielens-20M dataset is larger than the other two datasets and uses the transE method for knowledge graph embedding. Although transE is simple and efficient, it performs poorly when encoding complex relationships. Thus, the contextual information of the entities may bring the noise. Besides, on the Movielens-1M dataset, the performance of KPHAN increases consistently with the complexity of the fusion methods, indicating that transR can obtain more accurate knowledge graph embedding by projecting different relations into different spaces. The result indicates that it is vital to choose the appropriate knowledge embedding method to enhance item encoding, and also, different fusion methods can affect the model's performance.

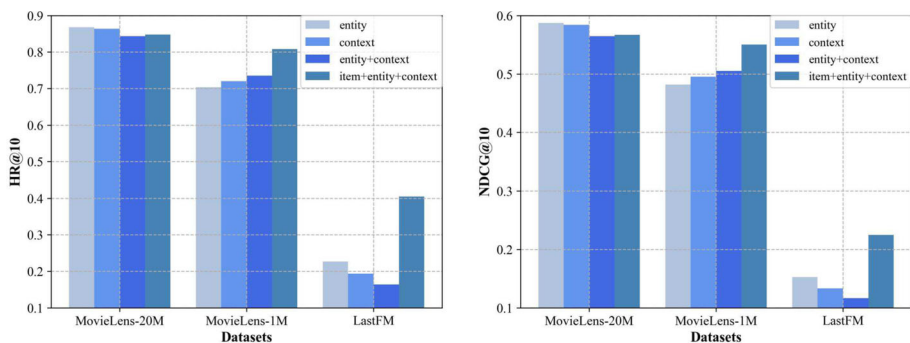


Figure 5 Effect of knowledge fusion methods

6 Conclusion

In this paper, we explore the effectiveness of item knowledge information and user long-term and short-term preferences in the performance of sequential recommendation and propose the KPHAN method. KPHAN adopts knowledge-enhanced item encoding to capture semantic associations between items and achieve more fine-grained item representation. In addition, we design a personalized hierarchical attention network to model dynamic short-term and static long-term user preferences. Experiments show that our method achieves state-of-the-art performance on three public datasets.

In the future, we will explore how to obtain the optimal combination of relations for first-order subgraphs. We also consider modeling higher-order semantic ties by adopting a graph attention network and designing data augmentation methods to construct positive and negative sequences and contrastive loss to implement a sequential recommendation model based on self-supervised learning.

Author Contributions Shuqi Ruan: Methodology, Software, Formal Analysis, Investigation, Data Curation, Writing-Original Draft, Validation. Chao Yang: Conceptualization, Writing-Original Draft, Project Administration, Resources, Funding Acquisition. Dongsheng Li: Supervision, Writing-Review & Editing.

Funding This work was supported in part by the National Natural Science Foundation of China under Grant 62172156 and 62072169, and the Natural Science Foundation of Hunan Province, China under Grant 2021JJ30152.

Availability of data and materials The datasets used or analysed during the current study are available from the corresponding author on reasonable request.

Declarations

Ethics approval This declaration is not applicable.

Competing interests The authors declare no competing interests.

References

1. Adomavicius, G., Tuzhilin, A.: Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Trans. Knowl. Data Eng.* **17**(6), 734–749 (2005)
2. Smith, B., Linden, G.: Two decades of recommender systems at amazon. *com. IEEE Internet Comput.* **21**(3), 12–18 (2017)
3. Kumar, S., Zhang, X., Leskovec, J.: Predicting dynamic embedding trajectory in temporal interaction networks. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1269–1278 (2019)
4. Hidasi, B., Karatzoglou, A., Baltrunas, L., Tikk, D.: Session-based recommendations with recurrent neural networks. In: *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2–4, 2016, Conference Track Proceedings* (2016)
5. Kang, W.-C., McAuley, J.: Self-attentive sequential recommendation. In: *2018 IEEE International Conference on Data Mining (ICDM)*, pp. 197–206. IEEE (2018)
6. Wang, S., Hu, L., Wang, Y., Cao, L., Sheng, Q.Z., Orgun, M.: Sequential recommender systems: challenges, progress and prospects. In: *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10–16, 2019*, pp. 6332–6338 (2019)
7. Rendle, S., Freudenthaler, C., Schmidt-Thieme, L.: Factorizing personalized markov chains for next-basket recommendation. In: *Proceedings of the 19th International Conference on World Wide Web*, pp. 811–820 (2010)

8. Fang, H., Zhang, D., Shu, Y., Guo, G.: Deep learning for sequential recommendation: Algorithms, influential factors, and evaluations. *ACM Transactions on Information Systems (TOIS)* **39**(1), 1–42 (2020)
9. He, R., McAuley, J.: Fusing similarity models with markov chains for sparse sequential recommendation. In: 2016 IEEE 16th International Conference on Data Mining (ICDM), pp. 191–200. IEEE (2016)
10. Kabbur, S., Ning, X., Karypis, G.: Fism: factored item similarity models for top-n recommender systems. In: Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 659–667 (2013)
11. Tang, J., Wang, K.: Personalized top-n sequential recommendation via convolutional sequence embedding. In: Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining, pp. 565–573 (2018)
12. Tuan, T.X., Phuong, T.M.: 3d convolutional networks for session-based recommendation with content features. In: Proceedings of the Eleventh ACM Conference on Recommender Systems, pp. 138–146 (2017)
13. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. *Advances in Neural Information Processing Systems* **30** (2017)
14. Zhang, T., Zhao, P., Liu, Y., Sheng, V.S., Xu, J., Wang, D., Liu, G., Zhou, X.: Feature-level deeper self-attention network for sequential recommendation. In: IJCAI, pp. 4320–4326 (2019)
15. Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., Jiang, P.: Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In: Proceedings of the 28th ACM International Conference on Information and Knowledge Management, pp. 1441–1450 (2019)
16. Cho, S.M., Park, E., Yoo, S.: Meantime: Mixture of attention mechanisms with multi-temporal embeddings for sequential recommendation. In: Fourteenth ACM Conference on Recommender Systems, pp. 515–520 (2020)
17. Zheng, L., Guo, N., Chen, W., Yu, J., Jiang, D.: Sentiment-guided sequential recommendation. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 1957–1960 (2020)
18. Wu, L., Li, S., Hsieh, C.-J., Sharpnack, J.: Sse-pt: Sequential recommendation via personalized transformer. In: Fourteenth ACM Conference on Recommender Systems, pp. 328–337 (2020)
19. Yuan, X., Duan, D., Tong, L., Shi, L., Zhang, C.: Icai-sr: Item categorical attribute integrated sequential recommendation. In: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 1687–1691 (2021)
20. Xie, Y., Zhou, P., Kim, S.: Decoupled side information fusion for sequential recommendation. Proceedings of the 45th International ACM SIGIR conference on Research and Development in Information Retrieval (2022)
21. Cai, W., Pan, W., Mao, J., Yu, Z., Xu, C.: Aspect re-distribution for learning better item embeddings in sequential recommendation. In: Proceedings of the 16th ACM Conference on Recommender Systems, pp. 49–58 (2022)
22. Rashed, A., Elsayed, S., Schmidt-Thieme, L.: Context and attribute-aware sequential recommendation via cross-attention. In: Proceedings of the 16th ACM Conference on Recommender Systems, pp. 71–80 (2022)
23. Wu, S., Tang, Y., Zhu, Y., Wang, L., Xie, X., Tan, T.: Session-based recommendation with graph neural networks. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 33, pp. 346–353 (2019)
24. Wang, J., Ding, K., Hong, L., Liu, H., Caverlee, J.: Next-item recommendation with sequential hypergraphs. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 1101–1110 (2020)
25. Chang, J., Gao, C., Zheng, Y., Hui, Y., Niu, Y., Song, Y., Jin, D., Li, Y.: Sequential recommendation with graph neural networks. In: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 378–387 (2021)
26. Pang, Y., Wu, L., Shen, Q., Zhang, Y., Wei, Z., Xu, F., Chang, E., Long, B., Pei, J.: Heterogeneous global graph neural networks for personalized session-based recommendation. In: Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining, pp. 775–783 (2022)
27. Guo, Q., Zhuang, F., Qin, C., Zhu, H., Xie, X., Xiong, H., He, Q.: A survey on knowledge graph-based recommender systems. *IEEE Trans. Knowl. Data Eng.* **34**(8), 3549–3568 (2020)
28. Wang, H., Zhang, F., Xie, X., Guo, M.: Dkn: Deep knowledge-aware network for news recommendation. In: Proceedings of the 2018 World Wide Web Conference, pp. 1835–1844 (2018)
29. Zhang, F., Yuan, N.J., Lian, D., Xie, X., Ma, W.-Y.: Collaborative knowledge base embedding for recommender systems. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 353–362 (2016)

30. Wang, X., Wang, D., Xu, C., He, X., Cao, Y., Chua, T.-S.: Explainable reasoning over knowledge graphs for recommendation. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 33, pp. 5329–5336 (2019)
31. Huang, X., Fang, Q., Qian, S., Sang, J., Li, Y., Xu, C.: Explainable interaction-driven user modeling over knowledge graph for sequential recommendation. In: Proceedings of the 27th ACM International Conference on Multimedia, pp. 548–556 (2019)
32. Wang, H., Zhang, F., Wang, J., Zhao, M., Li, W., Xie, X., Guo, M.: Ripplenet: Propagating user preferences on the knowledge graph for recommender systems. In: Proceedings of the 27th ACM International Conference on Information and Knowledge Management, pp. 417–426 (2018)
33. Wang, X., He, X., Cao, Y., Liu, M., Chua, T.-S.: Kgat: Knowledge graph attention network for recommendation. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 950–958 (2019)
34. Huang, J., Zhao, W.X., Dou, H., Wen, J.-R., Chang, E.Y.: Improving sequential recommendation with knowledge-enhanced memory networks. In: The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval, pp. 505–514 (2018)
35. Wang, P., Fan, Y., Xia, L., Zhao, W.X., Niu, S., Huang, J.: Kerl: A knowledge-guided reinforcement learning model for sequential recommendation. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 209–218 (2020)
36. Shani, G., Heckerman, D., Brafman, R.I., Boutilier, C.: An MDP-based recommender system. *Journal of Machine Learning Research* **6**(9) (2005)
37. LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. *Nature* **521**(7553), 436–444 (2015)
38. Hidasi, B., Quadrana, M., Karatzoglou, A., Tikk, D.: Parallel recurrent neural network architectures for feature-rich session-based recommendations. In: Proceedings of the 10th ACM Conference on Recommender Systems, pp. 241–248 (2016)
39. Wu, C.-Y., Ahmed, A., Beutel, A., Smola, A.J., Jing, H.: Recurrent recommender networks. In: Proceedings of the Tenth ACM International Conference on Web Search and Data Mining, pp. 495–503 (2017)
40. Li, J., Ren, P., Chen, Z., Ren, Z., Lian, T., Ma, J.: Neural attentive session-based recommendation. In: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, pp. 1419–1428 (2017)
41. Quadrana, M., Karatzoglou, A., Hidasi, B., Cremonesi, P.: Personalizing session-based recommendations with hierarchical recurrent neural networks. In: Proceedings of the Eleventh ACM Conference on Recommender Systems, pp. 130–137 (2017)
42. Tan, Y.K., Xu, X., Liu, Y.: Improved recurrent neural networks for session-based recommendations. In: Proceedings of the 1st Workshop on Deep Learning for Recommender Systems, pp. 17–22 (2016)
43. Yu, F., Liu, Q., Wu, S., Wang, L., Tan, T.: A dynamic recurrent model for next basket recommendation. In: Proceedings of the 39th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 729–732 (2016)
44. Li, J., Wang, Y., McAuley, J.: Time interval aware self-attention for sequential recommendation. In: Proceedings of the 13th International Conference on Web Search and Data Mining, pp. 322–330 (2020)
45. Lin, J., Pan, W., Ming, Z.: Fissa: fusing item similarity models with self-attention networks for sequential recommendation. In: Fourteenth ACM Conference on Recommender Systems, pp. 130–139 (2020)
46. Xu, C., Zhao, P., Liu, Y., Sheng, V.S., Xu, J., Zhuang, F., Fang, J., Zhou, X.: Graph contextualized self-attention network for session-based recommendation. In: IJCAI, vol. 19, pp. 3940–3946 (2019)
47. Bordes, A., Usunier, N., Garcia-Duran, A., Weston, J., Yakhnenko, O.: Translating embeddings for modeling multi-relational data. *Advances in Neural Information Processing Systems* **26** (2013)
48. Wang, Z., Zhang, J., Feng, J., Chen, Z.: Knowledge graph embedding by translating on hyperplanes. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 28 (2014)
49. Ji, G., He, S., Xu, L., Liu, K., Zhao, J.: Knowledge graph embedding via dynamic mapping matrix. In: Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (volume 1: Long Papers), pp. 687–696 (2015)
50. Lin, Y., Liu, Z., Sun, M., Liu, Y., Zhu, X.: Learning entity and relation embeddings for knowledge graph completion. In: Twenty-ninth AAAI Conference on Artificial Intelligence (2015)
51. Zhao, W.X., He, G., Yang, K., Dou, H., Huang, J., Ouyang, S., Wen, J.-R.: Kb4rec: A data set for linking knowledge bases with recommender systems. *Data Intell.* **1**(2), 121–136 (2019)
52. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. the 3rd International Conference for Learning Representations (2015)
53. Harper, F.M., Konstan, J.A.: The movielens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems (TIIS)* **5**(4), 1–19 (2015)

54. Bollacker, K., Evans, C., Paritosh, P., Sturge, T., Taylor, J.: Freebase: a collaboratively created graph database for structuring human knowledge. In: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, pp. 1247–1250 (2008)
55. Rendle, S., Freudenthaler, C., Gantner, Z., Schmidt-Thieme, L.: Bpr: Bayesian personalized ranking from implicit feedback. In: Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence, pp. 452–461 (2009)
56. He, R., Kang, W.-C., McAuley, J.: Translation-based recommendation. In: Proceedings of the Eleventh ACM Conference on Recommender Systems, pp. 161–169 (2017)
57. Ma, C., Kang, P., Liu, X.: Hierarchical gating networks for sequential recommendation. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 825–833 (2019)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.